ROBOTICS ENGINEERING AUDIT & TECHNICAL REPORT

CHINTU TWO-WHEELED BIPED ROBOT

Complete Repository Conflict Audit, Pin Collision Matrix & Field Deployment Guide

CRITICAL PHYSICAL WARNING: The two repository branches (`main` and `feature/chintu-defence-bot`) share hardware naming but feature completely clashing pinouts and incompatible electrical architectures. Flashing one branch's code on the other's wiring will cause immediate E-Stop loops, motor short circuits, or joint failure.

Primary Repository:	chintutherobo (branch: main)	**Secondary Branch:**	feature/chintu-defence-bot
Target Microcontroller:	Arduino Mega 2560 (ATmega2560)	**Target Host Supervisor:**	Raspberry Pi 4 / 5 (Linux)
Report Date:	November 9, 2026	**Audit Status:**	Verified & Resolved

1. High-Level Architecture Comparison

Both repositories are written for a two-wheeled bipedal robot with 3-link kinematic legs and microstepped wheel drives. However, they are fundamentally distinct engineering implementations:

Dimension	Main Production Stack (`chintutherobo`)	Feature Scaffold (`chintu-defence-bot`)
System Maturity	**PRODUCTION READY** Full-stack closed-loop balancing platform.	**STARTER SCAFFOLD** Bench unit test rig only.
Firmware Build Core	PlatformIO C++ (`robot/firmware/`) with `-O3` optimizations.	Arduino IDE (`.ino` + `.h`) in `arduino/` .
Balancing Controller	**200 Hz Active PID** + Pitch Estimator + Height Gain Scheduling.	**DISABLED** `VALIDATED = false` , gains set to 0.0.
Stepper Motor Generation	Hardware Timer3 & Timer4 CTC Interrupts (Up to **24,000 steps/s**).	Polled `AccelStepper` library in `loop()` (< 2,000 steps/s).
Servo Control (6x MG995)	Direct ATmega2560 Timer5 PWM Pins (Pins 44, 45, 46, 11, 12, 38).	External **PCA9685 I2C Board** over Pins 20/21.
Serial Protocol	Binary Framed Packets + CCITT-16 CRC @ **250,000 baud**.	ASCII text strings (e.g. `STATUS,42\n`) @ **115,200 baud**.
Supervisor Platform	FastAPI + WebSockets + Three.js 3D Web HUD (Pi 4/5).	Basic Python 3 blocking script logging to CSV (Pi Zero W).

2. Electrical & Pin Mapping Clashes (Fatal Hardware Risk)

This is the most critical conflict. If the physical wiring on the robot does not match the exact firmware flashed to the Arduino Mega 2560, hardware components will be damaged.

Mega Pin	`main` Function	`feature` Function	Electrical Collision Result
Pin 2	**Hardware E-STOP** (Input, Pull-up)	**STEP 1** (Motor 1 Step Pulse Output)	Pulses on Pin 2 trigger infinite E-STOP trips on Main; Grounding Pin 2 shorts driver output on Feature.
Pin 3	Unassigned / Free	**DIR 1** (Motor 1 Direction Output)	Motor 1 direction line left floating on Main.
Pin 4	**Global Stepper Enable** (Active LOW)	**STEP 2** (Motor 2 Step Pulse Output)	Step pulses toggle driver enable thousands of times per second; severe coil chatter and driver burnout.
Pin 5	**Left Stepper STEP** (Timer3 CTC)	**DIR 2** (Motor 2 Direction Output)	Frequency pulse interpreted as direction toggles; motor vibrates in place.
Pin 6	**Left Stepper DIR** (Digital Out)	**ENABLE 1** (Motor 1 Enable Output)	Direction bit cuts power to motor driver instead of setting forward/reverse.
Pin 7	**Right Stepper STEP** (Timer4 CTC)	**ENABLE 2** (Motor 2 Enable Output)	Pulse bursts enable and disable Motor 2 continuously.
Pin 8	**Right Stepper DIR** (Digital Out)	Unassigned / Free	Right wheel direction line unassigned on Feature.
Pin 11	**Right Hip Roll Servo** (Timer5 PWM)	Unassigned / Free	Right hip servo completely unresponsive on Feature.
Pin 12	**Right Hip Pitch Servo** (Timer5 PWM)	Unassigned / Free	Right thigh servo completely unresponsive on Feature.
Pin 20 / 21	MPU-6050 I2C (SDA / SCL)	MPU-6050 + **PCA9685 I2C Board**	Feature cannot move ANY servos unless PCA9685 expander is physically present on I2C bus.
Pin 38	**Right Knee Servo** (PWM)	Unassigned / Free	Right knee servo unresponsive on Feature.
Pins 44, 45, 46	**Left Hip Roll, Pitch, Knee Servos**	Unassigned / Free	Left leg servos completely unresponsive on Feature.
Pin A0	**Battery Voltage Divider ADC**	Unassigned / Free	Main reads 3S LiPo voltage for cutoff protection; Feature has no voltage sensing.

3. Communication & Serial Protocol Incompatibility

Connecting the Raspberry Pi backend from `main` to the Arduino Mega running `feature` (or vice versa) results in absolute communication breakdown:

- **Baud Rate:** `main` communicates at **250,000 baud**. `feature` communicates at **115,200 baud**.
- **Framing:** `main` uses a strict 8-byte framed binary packet with `0xAA 0x55` sync header, message sequence number, message type ID, variable payload, and 16-bit CCITT CRC checksum.
- `feature` sends plaintext ASCII terminated by newline: `STATUS,42\n`, `STAND,1\n`, `STOP,1\n`.
- **Result:** Neither side can decode a single frame from the other.

4. Inconsistencies & Legacy Artifacts Cleaned in `main`

During this comprehensive audit, three legacy contradictions inside the `main` repository were identified and resolved:

1. **Port 8000 Conflict (Resolved):** Port 8000 was completely freed from the supervisor, Web UI, scripts, and mock servers. The entire stack is now standardized on **Port 3456**.
2. **Web-UI WebSocket Crash Bug (Resolved):** Fixed an undeclared `protocol` variable reference in `robot/web-ui/js/api.js` line 79 that prevented the teleoperation command loop from initializing.
3. **Missing Pin 12 in Documentation (Resolved):** Added the missing `SERVO 5 (Pin 12) - Right Hip Pitch` row to the canonical wiring table in `robot/README.md`.
4. **Obsolete ESP32 Document (Deprecated):** Added a prominent warning banner to `README_RASPBERRY_PI.md` to clarify that Chintu runs on Arduino Mega 2560 + Raspberry Pi, not the obsolete ESP32/L298N prototype.

5. Step-by-Step Operator Runbook (Physical Deployment)

Phase 1: Flash Arduino Mega 2560 Firmware

```
cd robot/firmware
pio run -e megaatmega2560 --target upload
```

Phase 2: Supervisor Setup on Raspberry Pi

```
cd robot/backend
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# Create /etc/chintu.env with secure credentials:
sudo bash scripts/install_env.sh

# Start Supervisor Daemon:
python3 -m backend.server
```

Phase 3: Connect to Web Teleoperation HUD

- Connect your phone/laptop to the robot network:
  - **Hotspot Mode:** Connect to Wi-Fi `ChintuRobot_XXXX` → Open `http://192.168.4.1:3456`
  - **Local Wi-Fi Mode:** Open `http://<RaspberryPi_IP>:3456`
  - **Local PC Simulation:** Run `python robot/tools/mock_ws_server.py` → Open `http://localhost:3456`
- Enter your 32-character `ROBOT_AUTH_TOKEN` in the security modal.

Phase 4: Bench Stand Pre-Flight (Hawa Me Test)

> **MANDATORY TEST:** Always support the robot on a bench stand with wheels suspended off the ground for initial arming. Never place it directly on the floor without verifying wheel direction!

1. Check **Calibrate** tab in HUD: Ensure hip and knee joints are centered at neutral 90°.
2. Hold robot vertical ($0^\circ \pm 5^\circ$).
3. Perform **SLIDE TO ARM ROBOT >>** gesture on dashboard.
4. Hold stationary for 2.0 seconds while gyro bias calibrates → State transitions to `ARMED`.
5. Tilt robot forward: Wheels must spin forward to balance. Tilt back: Wheels must spin backward.

---

6. Final Engineering Verdict

- **Base Repository:** Keep `chintutherobo` (`main`) as the sole production baseline. It has the complete real-time balance loop, high-speed ISR stepper engine, and web teleoperation interface.
- **Scaffold Repository:** Keep `chintu-defence-bot` in its separate folder. Do not merge its files into `main`. Use its unit sketches (`arduino/tests/`) strictly for isolated bench checks of individual sensors, updating the pins to match `main`'s `config.h`.